BYTEGUARD

An eBPF/XDP-Based DNS Flood Mitigation System

Concurrency-safe per-source and aggregate admission control in the Linux datapath

Implementation, synchronization, laboratory evaluation and future architecture

Problem Statement: PS-022 — DNS query flood mitigation using eBPF/XDP

Category: Cybersecurity

Team: ByteGuard **Team code:** A3-PS022-TC127

Institution: Indian Institute of Information Technology, Design and Manufacturing, Jabalpur

Prashant Sengar · Janhvi Gokhale

Faculty mentor: Dr. Nitish Andola

31 October 2026 — final documentation revision

Evidence-qualified engineering report. Available historical measurements are identified explicitly; an absent final campaign is not replaced with projected results.

Table of Contents

1. Problem Description 3

2. Solution Proposed 4

3. XDP Packet Processing 5

4. Admission Control: Mathematics and State 6

5. Concurrency and Synchronization 7

6. BPF Map Design and Implementation Constraints 8

7. Testbed and Evaluation Methodology 9

8. Performance Evaluation 11

9. Throughput, Correctness and Concurrency Tests 15

10. Monitoring, Reproducibility and Deployment 16

11. Limitations and Open Engineering Questions 17

12. Proposed Improvements 18

13. Conclusion 22

14. References 22

Appendix A. Evidence Index and Reproduction 23

Appendix B. Statistics and Operational Commands 24

Appendix C. Release Readiness and Evidence Gaps 25

1. Problem Description

A DNS query flood can consume receive-path processing and DNS-server resources even when the physical link is not saturated. Filtering after the packet has traversed the networking stack can spend substantial work on requests that will ultimately be rejected. XDP supplies an earlier programmable receive-path decision point. Its placement and cost depend on native versus generic attachment and on the device; a virtual Ethernet experiment does not establish physical-NIC line-rate performance [1, 2].

For AIORI-3 PS-022, the engineering objective is to regulate excessive DNS query admission while retaining useful service for legitimate clients. Packet rejection, resource consumption and successful DNS responses are separate outcomes. A policy can reduce admitted load yet also reduce legitimate availability. ByteGuard is therefore evaluated as both an admission-control implementation and an experimental DNS mitigation baseline.

1.1 Threat model and engineering requirements

The controlled workload includes one high-rate source and a set of rotating or distributed source IPv4 addresses. Source addresses may be spoofed and are not identities. Multiple CPUs may execute the program concurrently. State must remain bounded, packet accesses must be verifier-safe, and failed multi-step admissions must not silently leak reserved global capacity.

The baseline does not claim to stop a saturated upstream link, authenticate DNS clients, prioritize legitimate users, or determine whether a queried name is malicious. It operates before normal Linux DNS processing and does not replace a resolver. All traffic-generation commands are confined to owned laboratory interfaces.

1.2 Evidence and revision boundary

Repository source is authoritative for implemented behavior. Raw experiment files are authoritative for reported measurements. This revision follows the supplied 14-page report's academic structure and visual style while correcting its obsolete insertion-race description and replacing its preliminary numerical appendix with an auditable evidence index.

The checkout contains 42 recorded experiments and 245 benchmark artifacts. Twelve clean schema-v4 runs form the historical before-tuning comparison; three later XDP-only diagnostic runs and one capture-enabled run are kept separate. No final Phase 2 campaign is present. Its 27-run profile is an execution plan, not completed evidence. User-reported later live validation is acknowledged, but no corresponding final successful-scrape or kernel-test log was supplied in this evidence snapshot.

2. Solution Proposed

2.1 Implemented baseline

ByteGuard combines shared per-source GCRA state with one global fixed-point token bucket. GCRA regulates one retained source history; the global bucket constrains aggregate admitted DNS traffic regardless of source count. BPF maps hold limiter state and per-CPU counters. A Go controller embeds the production object, loads maps/program, attaches XDP and exports map statistics through Prometheus. Grafana presents the resulting observations.

Component	Implemented mechanism	Actual configuration
Per-source state	Shared LRU hash, IPv4 source key	1024 entries
Source admission	64-bit GCRA TAT, bounded CAS	100 queries/s, burst 10, four attempts
Aggregate admission	Shared fixed-point bucket	1000 queries/s, burst 200
Global synchronization	One array value with BPF spin lock	One shared bucket
Statistics	Per-CPU array	12 unsigned 64-bit counters
Control/observation	Go, cilium/ebpf, custom Prometheus registry	Configurable interface, XDP mode and metrics address

These limits are configuration parameters. They are not measurements of maximum packet-processing capacity. The current policy has one aggregate admission class: it does not reserve capacity for clients judged legitimate.

2.2 Complete packet and observation flow

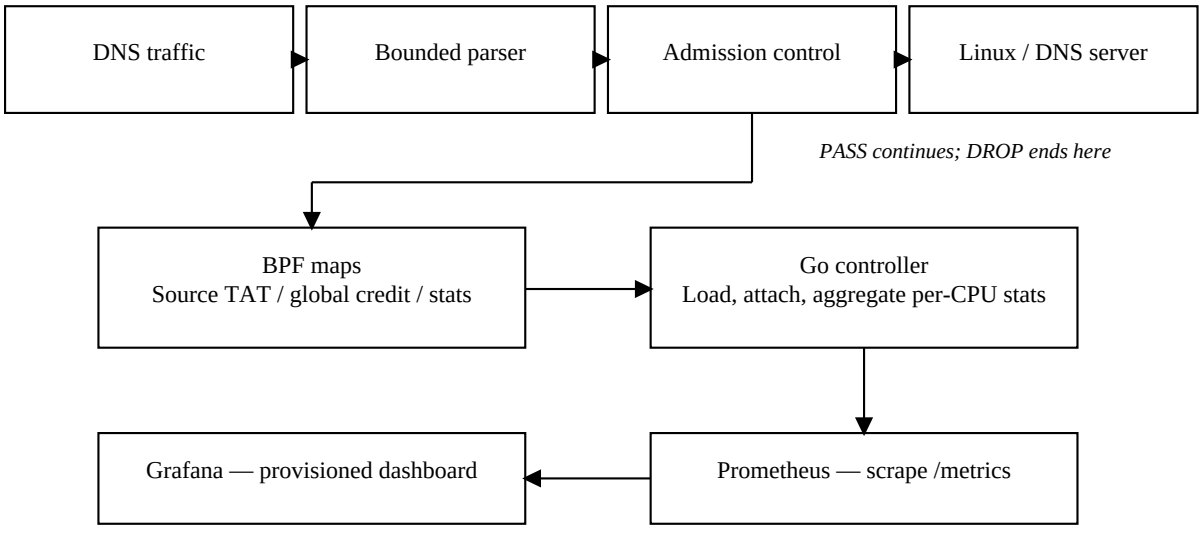


Figure 1. Implemented end-to-end architecture. The controller loads/attaches the program and reads the per-CPU statistics map; the packet verdict is produced in XDP. Only admitted packets continue to the DNS service.

Kernel counters describe recognized-query decisions; DNSPerf measures actual replies. Grafana reads real map statistics and does not convert PASS into client success. A valid demo must show increasing kernel counters and completed DNS responses.

3. XDP Packet Processing

3.1 Exact classification and bounds

`src/xdp/xdp_main.c` starts with an Ethernet header bounds check and accepts only the untagged IPv4 EtherType for further inspection. It validates the IPv4 version, minimum IHL, variable header extent, total-length consistency and packet end. Packets with the More Fragments bit or nonzero fragment offset bypass the DNS admission path. UDP protocol and header bounds are then checked.

UDP length must be at least eight bytes and equal the IPv4 payload length. Destination port must be 53. The program requires enough bytes for the fixed 12-byte DNS header, QR=0, standard opcode zero and QDCOUNT greater than zero. This is more specific than checking UDP/53 and the QR bit alone. It still does not parse or validate the actual question name, QTYPE or complete DNS message, and it does not verify IPv4/UDP checksums itself.

Every packet-derived header offset is checked before dereference. These checks establish access safety for the verifier; they do not establish full protocol validity. No unbounded DNS-name traversal or fragment reassembly occurs [3].

3.2 Bypass and final outcomes

Packet condition	Current action and accounting
Selected fixed-header IPv4 UDP/53 standard query	Enter source/global admission; increment DNS_QUERIES
Eligible and successfully committed query	XDP_PASS; increment DNS_PASS
Admission denial or handled terminal error	XDP_DROP; increment PACKET_DROP and one final reason
Truncated/invalid checked headers, non-UDP, other port	Early XDP_PASS; outside the DNS admission counters
QR=1, nonstandard opcode or zero questions	Early XDP_PASS; not charged as a selected query
IPv4 fragment, IPv6 or VLAN-tagged frame	Early XDP_PASS; no protection by this DNS limiter

The bypass behavior is explicit: malformed or unsupported packets are not automatically dropped. A public deployment must not mistake this selective baseline for a complete malformed-DNS firewall. DNS-over-TCP is outside its limiter scope. The program generates no DNS response, DNS Cookie, active challenge or TCP retry indication.

3.3 Attachment direction

XDP attaches to ingress on the server-facing veth endpoint. Attaching only to the client endpoint would observe responses arriving at the client rather than the outgoing queries targeted by the benchmark. The current harness requests native driver mode (`xdpdrv`) without automatic fallback. The Go controller accepts explicit native or generic selection; results from different modes must remain distinct.

4. Admission Control: Mathematics and State

4.1 Per-source GCRA

Let t be monotonic time in nanoseconds, T the stored theoretical arrival time, R_s the source rate and B_s the source burst. The implemented spacing and tolerance are:

$$I = \frac{10^9}{R_s} = 10^7 \text{ ns} = 10 \text{ ms}, \qquad \tau = (B_s - 1)I = 90 \text{ ms}.$$

A source is eligible if T is not ahead of t, or if its lead is within the tolerance. The code uses this difference form to avoid subtracting the tolerance from an unsigned timestamp:

$$\text{eligible}(T, t) = (T \le t) \text{ or } (T > t \text{ and } T - t \le \tau).$$

A successful commit advances the state by one interval:

$$T_{\text{new}} = \max(t, T_{\text{old}}) + I.$$

The commit verifies eligibility again against the current state, checks addition overflow and performs 64-bit compare-and-swap. Up to four unrolled attempts recompute from a newly read TAT. Ineligible packets and exhausted retries do not advance that TAT. At a fixed timestamp, a fresh retained source can admit ten queries; after 50 ms, five more intervals become available in the unit-test setup.

A new source is initialized with T=t+I and packet_seen=1 only after global capacity has been reserved. Existing successful admissions increment packet_seen atomically. The source value is 16 bytes: two unsigned 64-bit fields. Packet_seen is diagnostic state; it is not a qualification or trust score.

4.2 Global fixed-point bucket

The shared bucket stores a spin lock, credit C and last-refill timestamp. One token is represented by $q=10^9$ integer credit units. With global rate R_g=1000 tokens/s and burst B_g=200:

$$q = 10^9, \qquad C_{\max} = B_g q = 200 \times 10^9.$$

For positive elapsed nanoseconds d, the conceptual refill and reservation are:

$$C' = \min(C_{\max}, C + R_g d), \qquad C_{\text{reserved}} = C' - q \quad \text{if } C' \ge q.$$

The dimensions follow from the chosen fixed point: multiplying elapsed nanoseconds by tokens/second gives credit units when $q=10^9$. Fractional tokens therefore accumulate without floating point. All related fields are updated under the same spin lock.

4.3 Refill overflow and refunds

Before multiplying elapsed time, the implementation calculates time to fill using a ceiling division:

$$d_{\text{full}} = \left\lceil \frac{C_{\max} - C}{R_g} \right\rceil.$$

If elapsed time reaches that value, credit is set directly to the cap. Otherwise the bounded multiplication/addition is used. Non-increasing timestamps cause no refill and do not move the stored timestamp backwards. A terminal failure after reservation invokes a synchronized saturating refund:

$$C_{\text{refund}} = \min(C_{\max}, C + q).$$

Saturation means a refund call is not proof that one full token was added; concurrent refill may already have filled the bucket. GLOBAL_REFUND counts calls, not independently confirmed successful credits. The benchmark seeds a full bucket before attach. The Go controller uses ordinary first-packet refill of zero-initialized array state.

5. Concurrency and Synchronization

5.1 Shared source state and bounded atomic commit

A source budget must be shared across packet-processing CPUs. Independent per-CPU source limiters would multiply effective allowances or make behavior depend on receive steering unless a separate reconciliation design were introduced. A shared LRU value gives concurrent packets a common TAT while bounding memory.

A plain read/compute/store loses updates: two CPUs can read the same TAT and both overwrite it with the same successor while admitting two packets. CAS provides a linearization point for the chosen 64-bit TAT update. A competing update causes recomputation, not an unconditional overwrite. Four bounded attempts make worst-case work explicit and verifier-friendly. Exhaustion drops the packet and refunds its already reserved global token; the retry bound does not guarantee a successful commit or fair admission for every eligible packet.

5.2 Existing-source ordering

The exact path is: read-only source eligibility check; global reservation under the bucket lock; source commit with bounded CAS; refund and drop if commit fails. The initial read rejects an already ineligible source before taking global capacity. It is not a source reservation: another CPU may advance TAT immediately afterward. The final commit must recheck eligibility and may fail even after the initial check succeeded.

For example, CPUs A and B may both observe an eligible TAT and reserve separate global tokens. A commits first. B's CAS can fail, reread, and either commit another valid interval or terminate. If B terminates, returning its token avoids permanently charging a packet that was not admitted.

5.3 New-source insertion and races

The new-source path first reserves global capacity, then inserts initialized state using BPF_NOEXIST. This avoids creating fresh source state when the global budget is already unavailable. A successful insert records SOURCE_INSERT. The kernel's map operation decides whether the key is still absent [4].

On -EEXIST, the program records INSERT_RACE and re-looks up the entry inserted by another CPU. It attempts source_commit on that entry while retaining its existing global reservation. A successful commit admits and increments packet_seen. A missing re-lookup, failed commit or other insertion error refunds and drops, with the corresponding counters. Thus an insertion race is an event, not itself a guaranteed drop. This corrects the reference report's older statement that every losing insertion automatically refunds and drops.

5.4 Global locking and cross-map limits

Credit, refill time and reservation must form a consistent multi-field update, so the single global array value contains a BPF spin lock. The lock covers only bucket operations; parsing and source CAS execute outside it. Refunds take the same lock. An unlocked read-modify-write would allow lost reservations or inconsistent refill history.

The global reservation and source update are separate operations, not one atomic transaction across maps. Refund is compensation, not serializable cross-map rollback. Other packets can observe temporarily reserved capacity and be denied before a later refund. Successful decisions consume source history and global capacity under the implemented path, but there is no arrival-order fairness, client preference or formal whole-system concurrency proof.

LRU eviction can recreate source history. GCRA's retained-state argument is not a lifetime guarantee for an address under arbitrary churn. The global bucket remains the shared aggregate accounting point.

6. BPF Map Design and Implementation Constraints

Map	Type / capacity	Key and value	Purpose
source_map	BPF_MAP_TYPE_LRU_HASH, 1024	IPv4 u32; TAT u64 + packet_seen u64	Shared source history with bounded retention
global_map	BPF_MAP_TYPE_ARRAY, 1	u32 zero; spin lock + credit + timestamp	Shared aggregate reservation/refund state
stat_map	BPF_MAP_TYPE_PERCPU_ARRAY, 12	u32 ID; u64 per possible CPU	Low-contention decision/event counters

On the documented ABI, global_state is 24 bytes including lock alignment padding; the benchmark loader uses the matching binary layout. The map's complete allocated memory is larger than the sum of visible keys/values because of kernel bookkeeping, hash/LRU structures and per-CPU storage. No exact resident-memory measurement is available.

6.1 Why bounded LRU and fixed arrays

The LRU map caps source memory under address churn and avoids an unbounded userspace-managed allocation path. It can evict useful history, and its recency mechanism is not a reputation system or an exact policy-visible ordering of sources. The single-entry global array avoids eviction of the accounting anchor. Per-CPU statistics separate frequent increments from the shared admission synchronization [4].

6.2 Verifier and execution constraints

Packet accesses require proven data/data_end bounds. Loops and retry work must be bounded; the source CAS loop is explicitly unrolled. The global locked region is small and contains no ordinary map lookups after lock acquisition. Time is sampled before acquiring the lock. Integer arithmetic, state layout and BPF atomic/spin-lock support are kernel/toolchain requirements; merely compiling the C source is not proof of kernel verifier acceptance [3].

The build uses Clang's BPF target, optimization and v3 instruction selection. It does not define DEBUG in the current production recipe. Enabling bpf_printk changes instrumentation overhead and must not be mixed into a performance comparison without recording it. Test-only clock and failure maps exist only when BYTEGUARD_TEST is defined. The embedded production-object test checks their absence.

6.3 Statistics consistency

For a quiescent complete snapshot, recognized DNS queries equal DNS_PASS plus PACKET_DROP. Total drops equal the four final reason counters. SOURCE_HIT, SOURCE_INSERT, INSERT_RACE, MAP_ERROR and GLOBAL_REFUND are events rather than additional packet outcomes. Userspace reads each per-CPU array entry and sums all possible CPUs. Separate reads are not a globally atomic snapshot, so transient identity mismatches during active traffic are not themselves proof of a limiter error.

7. Testbed and Evaluation Methodology

7.1 Evidence selection and environment

The main comparison is explicitly the twelve clean before-tuning/schema-v4 records, not the newest name encountered on disk. Six groups cover three modes and two attack levels, with seeds 44 and 45 in every group. Each raw DNSPerf summary was reparsed and compared to saved sent/completed/lost/NOERROR counts and latency values. Attack logs were reparsed, attack PCAP hashes verified, and existing accounting/cleanup audits checked. All selected records pass these checks. This is retrospective evidence validation, not a new experiment.

Historical metadata records kernel 7.3.0-rc6-prashant-dev-00010-g69f80fef3153, 12 logical CPUs, Clang 22.1.8, bpftool 7.8.0, nftables 1.1.6 and tcpreplay 4.6.1. Raw logs identify DNSPerf 2.16.0 and dnsmasq 2.93. CPU model, affinity, governor and temperature were not saved in those runs. The current checkout preflight reports an AMD Ryzen 5 8645HS, but that later observation cannot retroactively establish the historical machine configuration.

7.2 Namespace topology and DNS service

The benchmark creates bgbench-a with bgc0 at 10.10.0.1/24 and bgbench-b with bgs0 at 10.10.0.2/24. The veth pair is moved wholly into those namespaces. DNSPerf and tcpreplay run in the client namespace. dnsmasq binds the server address and locally answers bench.test names with 10.10.0.2; it has no external upstream resolver or hosts-file dependency. XDP attaches to server ingress bgs0. nftables installs a netdev-ingress chain only in the owned server namespace. No-filter installs neither filter.

Fresh namespaces, source maps, global state and nftables rules are created per run. Readiness probes verify an actual local DNS response and warm DNS/ARP before filter setup. No additional timed warmup is implemented. Normal completion and handled exceptions attempt cleanup; stale state must be inspected and cleaned explicitly after a crash.

7.3 Workload and comparison conditions

Parameter	Historical comparison
Legitimate workload	www.bench.test A, 50 QPS, one real client source
Duration / timeout / outstanding	10-second offer window; 1-second DNS timeout; up to 1000 outstanding
Attack levels	0 or 3000 offered PPS
Attack pattern	1024 rotating source IPv4 addresses, standard UDP/53 A query
Packet recipe	1024 deterministic PCAP frames; attack.bench.test question
Repetitions	Two paired seeds, 44 and 45
Capture	Disabled in all twelve selected runs
Filtering	None; nft source/global limits; current XDP source/global design

The shared workload SHA-256 and XDP source fingerprint are recorded in Appendix A. The historical XDP source fingerprint matches the current protected source tree. Exact ELF hashes vary with build/debug paths and remain in individual records. Metadata names commit b1368ec2eb4157f9421b800fa3b423cf156302fc; the report does not assume that a commit identifier alone captures uncommitted code or toolchain differences.

7.4 Scheduling and measurement definitions

A pipe barrier releases the attack and legitimate generators close together; it does not synchronize first packet arrival. The attacker stops at the requested window boundary, before DNSPerf's response-timeout drain. CPU/interface snapshots bracket the offer window plus measurement/termination overhead, whose actual elapsed duration is recorded. Response outcomes are gathered after drain. Seeds repeat the same DNS/source recipe across modes; per-run Ethernet addresses necessarily match freshly created interfaces.

Legitimate success is NOERROR replies divided by client queries sent. Goodput is NOERROR replies for window-offered requests divided by the requested ten seconds. DNS latency is for completed responses only. dnsmasq CPU uses process ticks normalized to one core; host busy/SoftIRQ use all-core /proc/stat deltas and include generators and unrelated work. Device RX statistics are not an independent attack-generator rate or a verified throughput ceiling.

The nftables comparator uses a per-source meter with timeout 60 seconds and rate/burst 100/10, followed by a global rate/burst 1000/200. It matches UDP/53 more broadly than the XDP standard-query parser and uses a conventional limiter rather than identical GCRA, insertion and refund mechanics. Valid standard-query traffic makes the paths relevant comparators, not algorithmically identical policies.

8. Performance Evaluation

8.1 Main historical comparison

Table 1 is rebuilt from the twelve validated raw records. It is not a final Phase 2 result. Entries are arithmetic means of two trials, with latency representing the mean of the two per-trial answered-query means. Each trial emits 500 legitimate requests. Raw counts, individual latency values and variances are retained in trial-results.csv rather than hidden behind rounded means.

Mode	Attack PPS	Success (%)	Replies/s	Latency (ms)	Evidence
none	0	100.0	50.00	0.2355	H01, H02
nft	0	100.0	50.00	0.2270	H03, H04
xdp	0	100.0	50.00	0.2420	H05, H06
none	3000	100.0	50.00	0.1460	H07, H08
nft	3000	45.9	22.95	0.1595	H09, H10
xdp	3000	44.1	22.05	0.1860	H11, H12

Table 1. Historical before-tuning application outcomes. Replies/s is client-observed NOERROR goodput, not XDP PASS. Evidence IDs resolve to exact original run directories in Appendix A.

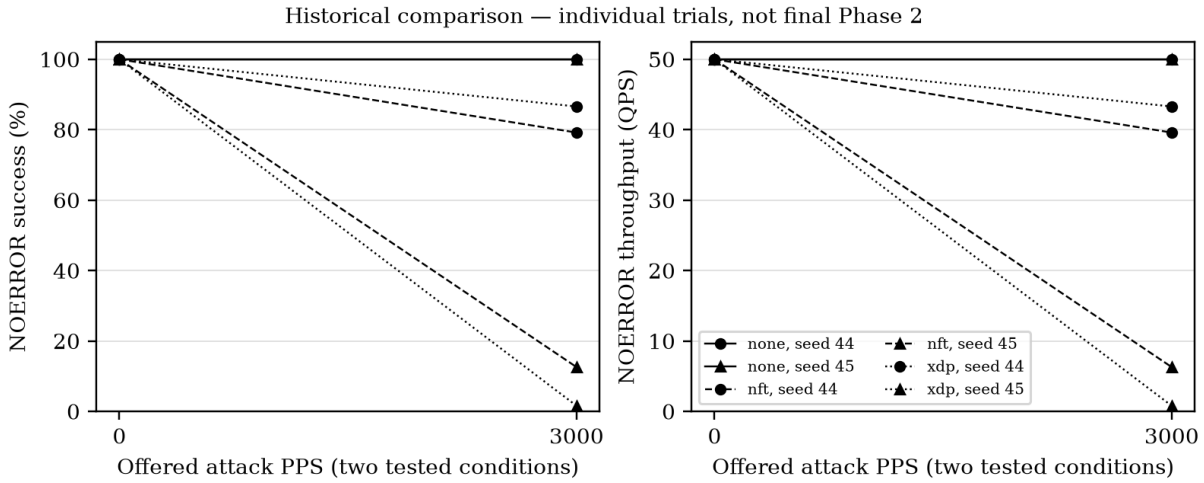


Figure 2. Historical legitimate success and throughput at the two tested attack conditions. Every point is an individual trial; markers/line styles distinguish modes and seeds. Connecting segments identify paired conditions, not a measured continuous scaling curve. Source: H01–H12.

All modes returned 100% legitimate success without attack. At 3000 offered distributed attack PPS, no-filter remained at 100% in both recorded trials, while nftables averaged 45.9% and XDP 44.1%. This workload does not demonstrate that the DNS server was overloaded or that the filters improved legitimate availability. It establishes that the selected filtering policy can impose substantial availability cost under the recorded offered load.

8.2 Variability and trial-level outcomes

ID	Mode	Seed	NOERROR / sent	Success (%)	Replies/s
H07	none	44	500 / 500	100.0	50.0
H08	none	45	500 / 500	100.0	50.0
H09	nft	44	396 / 500	79.2	39.6
H10	nft	45	63 / 500	12.6	6.3
H11	xdp	44	433 / 500	86.6	43.3
H12	xdp	45	8 / 500	1.6	0.8

Table 2. Historical distributed-attack trials. The two seeds produce sharply different outcomes for both filters; an unstable two-trial mean must not be presented as a robust performance ranking.

nftables success spans 12.6%–79.2%; XDP spans 1.6%–86.6%. Sample standard deviations are approximately 47.1 and 60.1 percentage points, respectively. With only two observations, neither the slight difference between the means nor the seed pairing establishes a reliable superiority claim. Seed changes, scheduling and competition can affect outcomes, but the artifacts do not identify their separate causal contributions.

8.3 Response latency and CPU observations

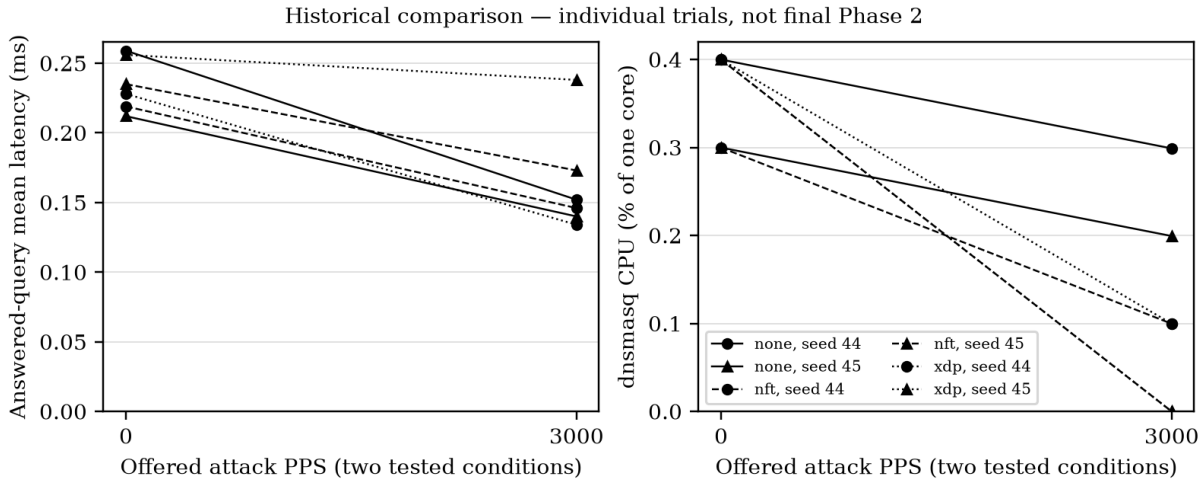


Figure 3. Historical answered-query latency and DNS-server CPU, individual trials. Latency excludes unanswered requests; low latency among a small admitted subset cannot offset poor overall completion. DNS CPU is quantized process-tick accounting, not filter-only CPU cost. Source: H01–H12.

Mode	Attack PPS	DNS CPU (%)	Host busy (%)	SoftIRQ (%)	Device RX PPS
none	0	0.350	12.53	0.249	50.7
nft	0	0.350	13.59	0.248	50.7
xdp	0	0.400	12.06	0.261	50.4
none	3000	0.249	19.17	0.297	3024.5
nft	3000	0.050	15.41	0.268	3020.9
xdp	3000	0.050	17.11	0.280	3021.6

Table 3. Historical system observations, two-trial means. DNS CPU is percent of one core; host busy and SoftIRQ are percent of aggregate CPU capacity. RX PPS is the server-device counter rate over its recorded measurement interval.

Several filtered trials record zero sampled dnsmasq CPU seconds or only one process-tick increment. These are finite-resolution samples, not proof that the server performed no work. The absolute server-CPU values are small and host-busy variation is much larger. CPU savings cannot be attributed to XDP execution alone: the host includes traffic generation and background activity, and many attack sources are spoofed with no normal response route. The artifacts do not prove how many admitted spoofed requests reached the DNS application.

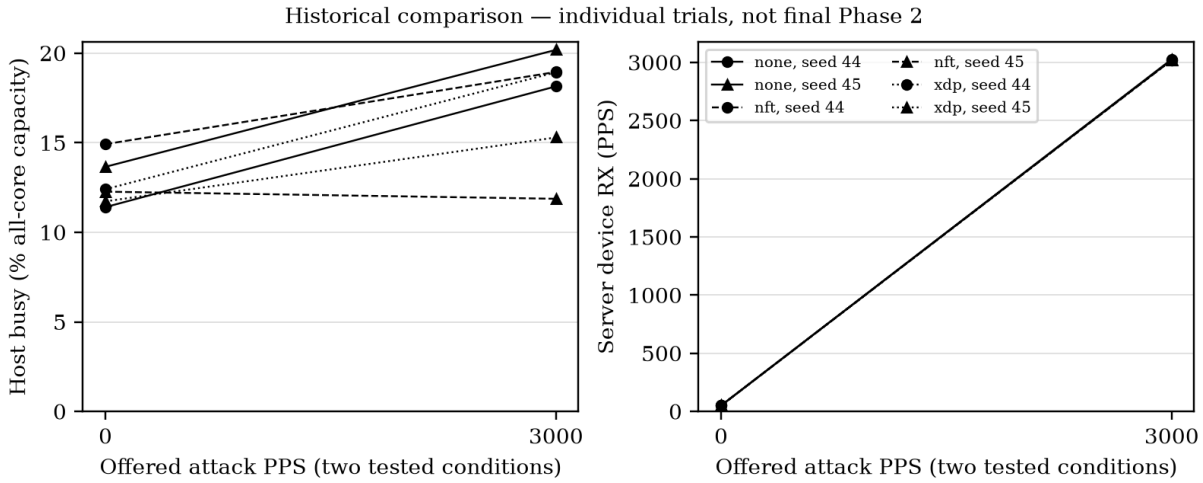


Figure 4. Historical host CPU and device ingress accounting. Individual trials expose background variation. Offered attack PPS comes from the replay command; RX PPS includes device-visible traffic and is not a measurement of maximum sustainable DNS service. Source: H01–H12.

8.4 Admission and drop behavior

ID	Mode	Admitted	Dropped	Source limit	Global limit
H09	nft	10141	20177	0	20177
H10	nft	10134	20160	0	20160
H11	xdp	10147	20189	0	20189
H12	xdp	10134	20161	0	20161

Table 4. Actual filter counters for the four historical distributed-attack trials. Counts combine legitimate and attack queries. Every recorded final drop in these rows is attributed to the global limiter; source-limit counts are zero. These counters do not identify which particular legitimate packets were denied.

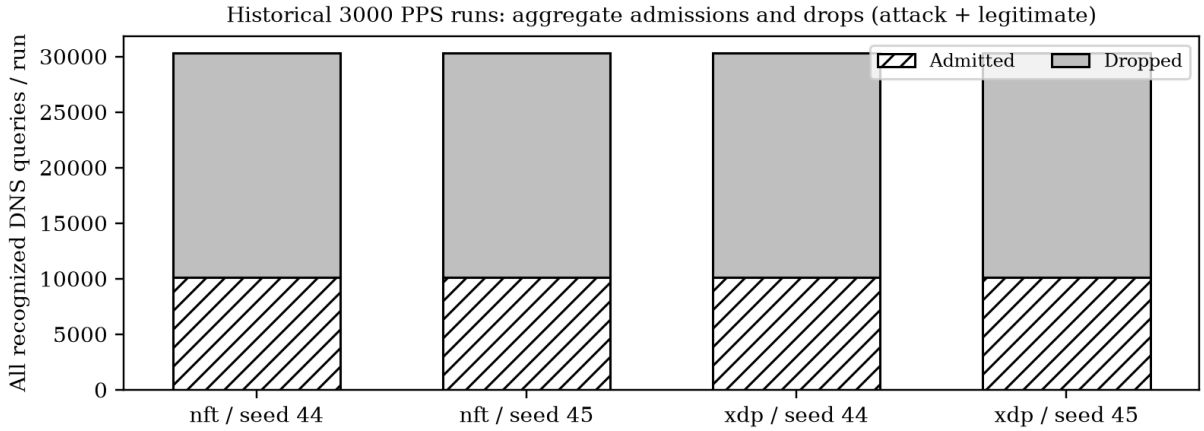


Figure 5. Historical aggregate filter outcomes, individual trials. Similar aggregate budgets can coexist with very different legitimate success. No-filter has no corresponding filter counters and is not assigned fabricated admission/drop values. Source: H09–H12.

The global budget is visibly active: approximately ten thousand queries are admitted and about twenty thousand are dropped in each filtered mixed run. This is consistent with competition for scarce aggregate

capacity as a mechanism for lost legitimate requests. It does not prove that every missing reply was a global-limit decision, nor explain all trial-to-trial variability. Packet-correlated evidence or additional instrumentation would be required for that causal claim.

XDP SOURCE_INSERT can exceed the map's 1024-entry capacity during a run; one recorded mixed trial has 3380 insertions. With 1024 attack addresses plus a legitimate address, repeated insertion is consistent with bounded-history churn. No explicit eviction counter exists, so the report does not reinterpret insertion events as directly measured eviction counts.

8.5 Separate diagnostic and capture evidence

The phase-check series consists of three XDP-only distributed runs, not a matched three-mode campaign. Their legitimate success values are retained below as diagnostic evidence and are not pooled with Table 1.

ID	Run suffix (see analysis.json)	NOERROR / sent	Success (%)
D01	20261027T215314-b9b2bd84	8 / 500	1.6
D02	20261027T215328-2b3ea82b	107 / 500	21.4
D03	20261027T215341-555c68de	69 / 500	13.8

Table 5. Separate XDP-only diagnostic runs, 500 legitimate requests each. They support discussion of variability, not a cross-mode ranking.

The mentor-evidence run enables bounded packet capture and records 81.2% legitimate success. Its client/server PCAPs are valuable for packet-path inspection, but capture changes overhead. It is excluded from the clean performance groups. Older schema-v3 and unversioned results are also excluded because their measurement windows and metadata are incompatible with the selected schema-v4 comparison.

8.6 Final campaign availability

The prepared Phase 2 matrix covers no attack, a single-source 3000 PPS attack and a 1024-source 3000 PPS attack, with three modes, ten seconds and three seeds: 27 planned runs. No such completed dataset is present in the audited archive. Consequently, no final single-source comparative table, no final three-trial ranking and no invented tail-latency series appear here. The historical results remain useful evidence of a policy trade-off, explicitly limited to their actual conditions.

9. Throughput, Correctness and Concurrency Tests

9.1 Throughput interpretation

Configured source/global rates are admission policy, not implementation throughput ceilings. The recorded 3000 PPS experiments observe behavior at one low laboratory load, not maximum sustainable packet rate. Legitimate throughput in Table 1 is based on actual NOERROR responses. Device RX and generated attack PPS are different measurements and must not substitute for DNS service capacity.

The reference PDF's 234698, 211315 and 166296 PPS development observations lack a corresponding complete raw test record in the supplied snapshot. They are therefore not retained as validated throughput comparisons. The repository's approximately 596 ns/packet note likewise lacks method and distribution and is excluded from performance conclusions. Both originals are preserved as exploratory material, rather than silently converted into final evidence.

9.2 Executed non-privileged validation

During this release preparation, `make controller test` rebuilds the controller and runs the C GCRA burst/recovery and 32-thread CAS tests, the Python benchmark/report suite, mocked demo lifecycle tests and Go race-enabled collector/object-layout tests. The saved release-validation record identifies the actual outcomes. Unit/report fixtures are temporary test data, not benchmark measurements.

The observed C concurrency trial admitted 10, rate-limited 22 and exhausted CAS zero times; the final TAT matched the sum of successful interval advances. The test accepts bounded success and validates state accounting; it does not prove progress, fairness or all possible kernel interleavings. The Go tests exercise aggregation through the real StatsReader, collector registration and HTTP handler using an explicit fake map, and verify HTTP 500 on a map-read failure.

9.3 In-kernel tests and claim limits

The repository has twelve kernel functional cases plus separate concurrent-accounting and LRU-history stress cases. Test-only clock/fault maps drive parser bypasses, burst/recovery, global exhaustion/refill, insertion collisions/errors and reserve/commit/refund branches. Injected races give branch coverage, not proof that an identical scheduling race occurred naturally. Stress mode checks concurrent accounting and deliberate source-history recreation after LRU churn.

These privileged tests are not rerun automatically for documentation. Earlier saved preparation records were blocked by interactive sudo, while the team reports subsequent live functional validation. No final kernel-run log is bundled to justify a new numerical pass count. The reference's older 8/8 claim is therefore superseded by the current test inventory and explicit evidence status, not repeated as a current run result.

10. Monitoring, Reproducibility and Deployment

10.1 Build, loading and lifetime

`make controller` compiles the production object, rebuilds on XDP header changes, copies it to the Go embed path and emits build/byteguard. The two historical tracked executables under src/controller are not the recommended launch artifacts. Startup logs include the embedded object's SHA-256 for comparison with the newly compiled object.

The Go controller requires an interface name, resolves its index, validates native/generic mode and reserves its configurable HTTP address before loading or attaching. It removes the BPF memlock restriction where needed, parses the embedded collection with cilium/ebpf, loads maps/program, validates the 12-entry per-CPU statistics map and performs an initial read. It attaches xdp_main using an XDP link and keeps the collection alive until termination.

SIGINT/SIGTERM stop HTTP with a bounded shutdown and release the XDP link and collection through deferred cleanup. Startup/runtime server failures return errors through the same cleanup path. The program does not persist or pin its normal controller maps; restarting gives fresh counters/state. The benchmark uses its own explicit bpftool loading/pinning lifecycle and does not depend on the monitoring controller.

10.2 Map aggregation and exporter

StatsReader uses the number of possible CPUs, reads a per-CPU vector for each counter and sums it. A custom collector emits twelve counter families. Its private registry is the same registry served by the /metrics handler. A failed map read emits an invalid metric, causing a failed scrape instead of a successful empty metrics response. Per-CPU values are summed, not averaged.

Prometheus samples the controller every two seconds. Rate graphs use rate() over a suitable range to handle counter growth and reset on restart. Grafana's provisioned datasource uses http://prometheus:9090 inside the Compose network; its eight-panel dashboard presents query/admission/drop rates, source/global drops, totals, errors, map/refund events and scrape health.

The ByteGuard dashboard reads observed kernel statistics through the controller. No client latency, legitimate success or isolated XDP CPU metric is fabricated. Those come from benchmark clients and system sampling. The saved earlier monitoring API check proves provisioning and Grafana-to-Prometheus queries but records a DOWN controller target. Later successful live monitoring is team-reported; publishing its actual nonempty scrape/query log remains an evidence-completion item.

10.3 Demo network and portable Compose

The foreground demo creates bgdemo-a/bgdemo-c connected to host endpoint bgdemo-s, using an isolated 10.203.0.0/24 subnet by default. dnsmasq binds only the lab server address; the controller stays in the host network namespace so Docker can reach its HTTP port. A different unused subnet can be selected explicitly. The script refuses name/route conflicts and stops owned processes/interfaces on normal signals. SIGKILL or power loss requires inspected manual recovery.

Compose mounts the case-sensitive Prometheus.yml, maps host.docker.internal to the Linux host gateway, and provisions Grafana from repository JSON/YAML. Default local UI ports are 9091 and 3001; metrics listen at 9090. The HTTP listener must be reachable from the Docker bridge. The benchmark's fully namespaced server is a different network topology and is not automatically reachable through a host HTTP port.

WSL2/Docker Desktop adds a distinct host/guest boundary. Its gateway may refer to Windows rather than the WSL guest, so a locally configured scrape target may be necessary. No developer-specific WSL address is embedded in the default setup. Linux with working BPF/XDP/namespaces is the reference deployment; no native macOS or Windows XDP support is claimed.

11. Limitations and Open Engineering Questions

- **Availability:** aggregate limiting supplies no legitimate-client preference. Distributed traffic can win tokens while legitimate queries fail, as the historical mixed trials show.
- **Identity and retention:** source IPv4 is not authentication. Spoofing, NAT and LRU churn complicate per-client interpretation; forgotten history can be recreated.
- **Protocol scope:** IPv6, VLAN tags, fragments, TCP DNS and unsupported/invalid classifications bypass this selective DNS admission path. The program does not parse question names or validate cookies/checksums.
- **Concurrency:** CAS protects one TAT word; the global lock protects bucket fields. Cross-map actions remain separate, bounded retries can deny under contention, and fairness is not guaranteed.
- **Observation:** snapshots are not globally atomic. Filter counters are aggregate decisions, not per-client response correlation or attack identity.
- **Measurement:** two historical repetitions, one machine, veth interfaces, no CPU isolation, missing historical CPU model and no saved raw tick samples limit causal conclusions. Newer harness code preserves raw samples; it cannot reconstruct absent old samples.
- **CPU cost:** server process ticks and host busy capacity do not isolate XDP cycles, lock contention or packet-generator overhead. Tiny quantized server-CPU differences need longer controlled measurements.
- **Latency:** only answered-request mean/min/max are available. Neither p95/p99 nor a timeout-inclusive service distribution was measured.
- **Deployment:** root/capabilities, verifier acceptance, mounted bpffs for the benchmark, native veth XDP support and compatible Go/Clang dependencies remain operational prerequisites.
- **Evidence completeness:** no final Phase 2 dataset, current privileged test log or final successful kernel-to-Grafana scrape is bundled. The current documentation records these limits instead of inventing substitutes.

The main engineering question is how to allocate a constrained budget so that legitimate completion improves without invalidating accounting or creating an easily abused trust shortcut. This requires new signals and controlled evaluation, not merely stronger packet dropping.

12. Proposed Improvements

All mechanisms in this section are research/design candidates. They are not implemented in the current ByteGuard baseline and have no measured performance improvement. Existing per-IP GCRA and the overall global-budget concept are the starting point; their replacement is not implied. Prefix lengths, thresholds, partition sizes, promotion rules and final execution order remain undecided.

12.1 Subnet-level limiting

An additional prefix-keyed limiter could aggregate related addresses and reduce the benefit of rotating source IPs within a network. Its interaction with per-IP GCRA must be defined: a source can be individually eligible yet belong to an exhausted prefix. Prefix selection should be driven by deployment and collateral-damage measurements, not fixed here as an arbitrary universal length.

Many unrelated legitimate clients can share a NAT address or prefix. Membership in a subnet is not evidence of abuse; coarse aggregation can punish entire organizations. The design needs bounded map capacity, clear idle expiration, churn behavior and tested reservation/refund relationships with source and global state. Prefix-limiter rejection should be reported separately so availability losses can be attributed rather than hidden in one aggregate drop counter.

12.2 Qualified-source history

A bounded non-LRU map could retain selected sources whose historical signals justify a carefully limited admission category. A non-LRU choice avoids automatic displacement of qualified state by arbitrary new addresses, but capacity exhaustion then requires an explicit admission, replacement or expiration policy. It is not a reason to allocate an unbounded allowlist.

Candidate signals include repeated observation over time, historical rate-limit behavior, query-pattern consistency and response-derived reachability information where available. Simply passing the current limiter must never automatically confer trust: attackers can pass during quiet periods or win scarce tokens. An address's past behavior does not bind a future packet to the same actor, especially with spoofing, reassignment or NAT.

Promotion, refresh, expiration, demotion and eviction must be measurable and abuse-resistant. Attackers may slowly accumulate history, poison qualification state or consume its bounded slots. Qualification should therefore remain revocable evidence with visible reasons, not a permanent bypass of all limits.

12.3 DNS Cookies and response-derived validation

DNS Cookies provide a protocol-supported mechanism involving client and server cookies, with limited protection against off-path spoofing. A validated server-cookie exchange can supply a stronger return-reachability signal than a source address observed only in requests [8, 9]. Merely finding an EDNS COOKIE field does not validate its authenticity.

A candidate design needs compatible DNS clients/server behavior and authenticated cookie verification coordinated with server secrets or a suitable trusted userspace component. Secret lifecycle, server pools and expiry/replay policy must be designed. The XDP fast path cannot safely be extended by treating an arbitrary option offset as a verified cookie, and the current code performs no such parsing or validation.

Clients without cookie support need a bounded discovery path. NAT, address changes and heterogeneous implementations affect compatibility and renewal. A captured or replayed valid signal and an attacker able to receive replies remain concerns: address reachability is not proof that future queries are benign. Response-based evidence also requires an observation point such as server integration, userspace logs or a separately designed egress hook; current server-ingress query counters do not observe outgoing DNS responses.

12.4 Query familiarity and userspace profiling

A userspace-assisted learner could maintain a bounded set of QNAME/QTYPE profiles and publish compact, fresh hints in a BPF-accessible map. The fast path might consult a carefully bounded normalized key or digest rather than an unbounded string structure. The population path, update rate, collisions, expiry and map exhaustion policy must be explicit and auditable.

DNS labels, compression pointers, unusual encodings and multiple questions make name extraction a substantive parsing problem. A bounded traversal budget and an explicit treatment of unsupported encodings are necessary for verifier safety and predictable work. The current fixed-header parser does not provide these facilities.

Popularity and familiarity are not legitimacy. Attackers can replay common names to occupy protected classes, and new legitimate queries can be unfamiliar. Learner poisoning, changing cache popularity and privacy of recorded query data must be evaluated. Familiarity should contribute limited evidence to classification, not independently grant admission or imply maliciousness when absent.

12.5 Conceptual layered classification

One possible explanatory sequence is: bounded parsing; per-source admission checks; subnet checks; selected source-history consultation; query-familiarity consultation; validated cookie evidence where supported; category selection; and enforcement of the overall traffic budget. This sequence is conceptual, not a finalized fast-path ordering.

In particular, read-only classification must be separated from state-changing commits. If one layer consumes a source/subnet/class allowance before another denies the packet, the design needs a consistent compensation or commit strategy. Category selection, global reservations, class borrowing and concurrent source promotion cannot simply be appended to the current path without rederiving accounting and testing its failure branches.

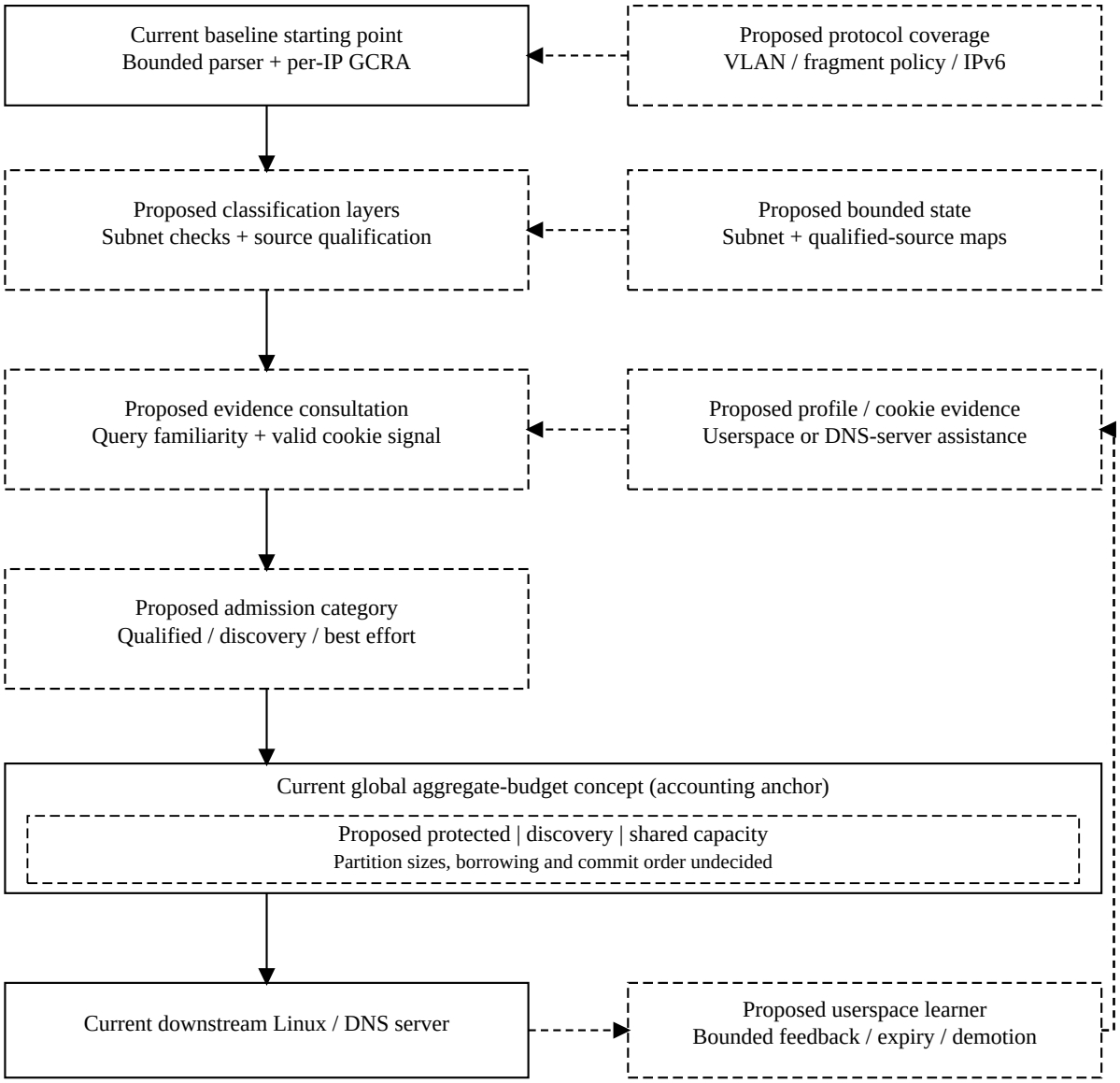
12.6 Protected, discovery and shared capacity

The proposed global budget could contain protected capacity for appropriately qualified traffic, limited discovery capacity for new or uncertain traffic, and shared/best-effort capacity. The aim is to keep distributed unqualified traffic from consuming every opportunity needed by established legitimate clients while still allowing new clients to establish useful evidence.

A hard aggregate budget must bound the sum of final admissions. Logical class balances may be subdivisions or reservations under that accounting anchor; they must not create independent unlimited buckets whose rates add beyond the intended cap. No numeric allocation is selected in this report.

Borrowing unused reservations may improve utilization, but it must specify whether and when borrowed credit can be reclaimed. Never lending leaves capacity idle; unlimited lending can erase the protection when it is needed. Discovery starvation, classification mistakes and abuse of qualified capacity require explicit minimum opportunities, expiry/demotion and overload behavior. A mistaken promotion should not monopolize an entire reserved class.

Concurrent classification and reservation must have one well-defined accounting model. Class changes, failed CAS, insertion races, borrowing and saturated refunds require tests for double charging, double refunds, leakage and aggregate overshoot. This proposal may improve availability under some distributed attacks; that hypothesis remains unmeasured until implementation and comparison against the unchanged baseline.



Solid outline: implemented baseline component or concept.

Dashed outline/arrows: proposed layer, extension or feedback path; no measured improvement claimed.

Figure 6. Proposed future architecture — not implemented in the current ByteGuard baseline. Solid boxes identify baseline components/concepts; dashed boxes and arrows show candidate state, classification and feedback. Layout is conceptual and does not prescribe a finalized commit/reservation order.

12.7 Protocol-coverage extensions

VLAN support should explicitly parse one or more tags with a bounded tag count, validate each header boundary and preserve the final network-protocol decision. Stacked tags expand offset and policy cases; the current EtherType check does not process them.

IPv4 fragments often lack transport headers, especially noninitial fragments. Options include a consistent drop/pass policy, bounded association state with expiry, or reassembly outside XDP before deeper validation. None is universally correct: pass-through can permit bypass, while coarse fragment dropping can damage legitimate traffic. Overlapping fragments, tiny fragments, state exhaustion and incomplete associations must be tested. Full reassembly is not currently implemented and must not be implied by a fragment classifier.

IPv6 is a separate extension involving address-key size, extension-header traversal and fragmentation rules. It should have an independently bounded parser and policy evaluation rather than being described as a simple additional EtherType.

12.8 Incremental future evaluation plan

Variant	Purpose	Required controlled comparison
Current baseline	Fixed reference	Freeze source, rates, mode and workload
Baseline + subnet limiter	Address-rotation control	Measure legitimate clients sharing prefixes and NAT
Baseline + qualification	Historical signal utility	Test promotion poisoning, expiry and new-client service
Baseline + profiling/cookies	Signal quality and cost	Separate profiling, authentic cookie validation and unsupported clients
Full layered/class budgets	Combined allocation	Verify aggregate accounting, discovery starvation and class abuse

Introduce one mechanism at a time and repeat matched no-attack, single-source, distributed, churn and shared-network workloads. Report legitimate completion, goodput, timeout-inclusive latency where instrumented, server/host/filter cost, map memory and decision reasons. Only after individual effects and failure paths are understood should combinations be evaluated. This is a research plan, not a submission-time implementation commitment.

13. Conclusion

ByteGuard implements bounded early DNS-query admission using shared per-source GCRA, a synchronized fixed-point global token bucket and compensating refunds. The current code explicitly handles source creation races by re-lookup and CAS, uses bounded work and exports twelve per-CPU counters through a Go/Prometheus/Grafana observation path. Reproducible demo, build and benchmark commands accompany the source.

The available historical comparison demonstrates successful no-attack DNS service, active aggregate filtering and a significant legitimate-availability trade-off during distributed traffic. No-filter preserves all 500 legitimate responses in both mixed trials; nftables and XDP have widely varying and lower completion. Earlier packet rejection is not equivalent to better legitimate-query success, and the current evidence does not justify a universal performance ranking, line-rate claim or isolated CPU-saving claim.

The final campaign is absent from this checkout and is not fabricated. The report provides raw evidence, hashes, individual results and reproduction commands so subsequent measurements can be incorporated transparently. Future subnet/history/profile/cookie signals and class-reserved capacity are clearly separated from the implemented baseline and require incremental correctness and availability evaluation.

14. References

[1] Linux kernel documentation. BPF/XDP overview. https://docs.kernel.org/bpf/

[2] T. Høiland-Jørgensen et al. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. ACM CoNEXT, 2018. https://doi.org/10.1145/3281411.3281443

[3] Linux kernel documentation. eBPF verifier. https://docs.kernel.org/bpf/verifier.html

[4] Linux kernel documentation. BPF_MAP_TYPE_HASH, with PERCPU and LRU variants. https://docs.kernel.org/bpf/map_hash.html

[5] Cilium eBPF project. Go library for loading, managing and attaching eBPF programs. https://github.com/cilium/ebpf

[6] Prometheus client_golang and Grafana provisioning documentation. https://github.com/prometheus/client_golang ; https://grafana.com/docs/grafana/latest/administration/provisioning/

[7] Team ByteGuard repository, source and preserved October 2026 laboratory artifacts. https://github.com/aiori-hackathon/A3-PS022-TC127 ; see Appendix A and reports/evidence/.

[8] D. Eastlake 3rd and M. Andrews. RFC 7873: Domain Name System (DNS) Cookies, 2016. https://www.rfc-editor.org/rfc/rfc7873

[9] O. Sury et al. RFC 9018: Interoperable Domain Name System (DNS) Server Cookies, 2021. https://www.rfc-editor.org/rfc/rfc9018

[10] AIORI-3 PS-022 and the team's Level 1 proposal, A3-PS022-TC127_ByteGuard_Proposal.pdf. The supplied fourteen-page ByteGuard_AIORI3_Technical_Report_Revised.pdf is the visual/coverage reference; its hash is recorded with this revision's source provenance.

Appendix A. Evidence Index and Reproduction

A.1 Frozen archive and source identity

reports/evidence/raw-benchmarks.tar.gz preserves exact bytes of all 245 local benchmark files at the release audit. artifact-manifest.json lists paths, lengths and SHA-256 values; run-catalogue.csv classifies all 42 runs. analysis.json, trial-results.csv and grouped-results.csv are regenerated from raw summaries/logs, not transcribed screenshots. The archive also preserves historical plots and binaries as evidence, not as recommended deployable executables.

The shared legitimate-workload SHA-256 is:

```
1a3f57f022f0da55196b0bd90265a925daa9abf524eb2c73f7fe25439851f71f
```

The current and historical selected XDP source fingerprint is:

```
1bfa75d16a9dd52b178c34033aa02fa4fde6eb501d7a66d1972a3ab7170bc94b
```

The fingerprint hashes the sorted XDP C/header names and bytes. Per-file protected-source hashes and the report-reference hash accompany the release audit. The matching source fingerprint does not eliminate toolchain, debug-path or runtime-environment uncertainty.

ID	Original run directory (under build/bench/runs/)
H01	before-tuning__none__none__0pps__50qps__20261027T192937-8d0dc8ce
H02	before-tuning__none__none__0pps__50qps__20261027T193011-8dc626c5
H03	before-tuning__nft__none__0pps__50qps__20261027T192948-2741b227
H04	before-tuning__nft__none__0pps__50qps__20261027T193021-3a1ad720
H05	before-tuning__xdp__none__0pps__50qps__20261027T192958-68a72e26
H06	before-tuning__xdp__none__0pps__50qps__20261027T193032-c69c3669
H07	before-tuning__none__distributed__3000pps__50qps__20261027T193046-1eaac51e
H08	before-tuning__none__distributed__3000pps__50qps__20261027T193122-c4be96dc
H09	before-tuning__nft__distributed__3000pps__50qps__20261027T193058-b8c7c09c
H10	before-tuning__nft__distributed__3000pps__50qps__20261027T193133-516eead9
H11	before-tuning__xdp__distributed__3000pps__50qps__20261027T193110-b496939f
H12	before-tuning__xdp__distributed__3000pps__50qps__20261027T193145-26e31918

Table A1. Main comparison evidence IDs. Within the archive, prefix each listed directory with runs/ and inspect summary.json, dnsperf.log, dnsmasq.log, optional attack.log/attack.pcap, experiment.toml and filter artifacts. Trial CSV includes each summary's exact hash.

A.2 Regenerate this document without networking

```
python3 -m venv --system-site-packages /tmp/byteguard-report
/tmp/byteguard-report/bin/pip install -r reports/source/requirements.txt
/tmp/byteguard-report/bin/python scripts/report/prepare_evidence.py
/tmp/byteguard-report/bin/python scripts/report/build_report.py
```

Install system Liberation fonts for the closest typography match; the renderer provides a standard Times fallback. Matplotlib uses a headless backend. The editable narrative is reports/source/report.md and the PDF renderer is scripts/report/build_report.py. Dependencies may require initial Internet access; rendering an already provisioned environment does not send traffic or require root. Intermediate PDF page images belong under reports/.build/.

Appendix B. Statistics and Operational Commands

ID	Kernel statistic / Prometheus suffix	Interpretation
0	DNS_QUERIES / dns_queries_total	Recognized queries entering admission
1	SOURCE_HIT / source_hits_total	Existing-source lookup events
2	SOURCE_INSERT / source_inserts_total	Successful new source insertions
3	INSERT_RACE / insert_races_total	BPF_NOEXIST collisions; not necessarily drops
4	MAP_ERROR / map_errors_total	Handled source-map errors
5	PACKET_DROP / packet_drops_total	Final admission-path drops
6	DNS_PASS / dns_pass_total	Final admitted selected queries
7	DROP_SOURCE_LIMIT / drop_source_limit_total	Initial source eligibility denials
8	DROP_GLOBAL_LIMIT / drop_global_limit_total	Failed global reservations
9	DROP_SOURCE_COMMIT / drop_source_commit_total	Terminal post-reservation source-commit denial
10	DROP_INTERNAL_ERROR / drop_internal_error_total	Other handled terminal failures
11	GLOBAL_REFUND / global_refunds_total	Refund calls, not confirmed credit additions

Table B1. All metric names have the byteguard_ prefix. These are counters, reset with new map state; rate() is appropriate for per-second charts.

```
make controller
make test
sudo make test-core test-core-stress
sudo python3 scripts/demo.py run
# In other terminals while the demo is running:
sudo python3 scripts/demo.py query --qps 50 --duration 10
sudo python3 scripts/demo.py flood --scenario single --pps 3000 --duration 15
make monitoring-up
curl --fail http://localhost:9090/metrics
# Ctrl+C in the demo terminal cleans its normal owned resources.
make monitoring-down
```

Grafana defaults to http://localhost:3001 and Prometheus to http://localhost:9091. Configurable demo credentials are documented in .env.example. An UP target alone is insufficient: confirm a nonempty ByteGuard query, increasing counters under traffic and actual client responses. Do not attach to an unrelated interface merely to test documentation.

Appendix C. Release Readiness and Evidence Gaps

The release inventory distinguishes essential source and test inputs from generated objects, tracked historical executables and cached outputs. No source, historical report, raw measurement or stash is deleted. The two tracked controller executables remain a maintainer approval item for untracking while preserving local copies. A repository-wide license also requires a maintainer decision; the kernel object's GPL string does not substitute for it.

The final submission includes the PDF and editable source, monochrome figures, exact evidence archive/manifest, updated README, architecture/setup/benchmark guides, cleanup inventory and release checklist. The source tree remains the baseline; the future diagram is explicitly a proposal.

Evidence still needed to claim a completed final campaign: matched no-filter/nft/XDP runs for all selected conditions, complete metadata and raw logs, verification of cleanup/accounting, and final trial-level analysis. Evidence still needed to strengthen the live-demo claim: saved successful kernel test output, attachment identity, before/after real counters, nonempty Prometheus custom queries and the corresponding Grafana observation. These gaps do not erase the existing recorded experiments; they bound what this revision can independently substantiate.

C.1 Reproduce measurements as new experiments

To repeat the historical workload recipe, use a new tag so old measurements remain distinct:

```
sudo python3 scripts/bench/harness.py matrix \
  --config configs/bench/distributed-demo.toml --tag reproduction-01
sudo python3 scripts/bench/harness.py verify --tag reproduction-01
sudo python3 scripts/bench/harness.py report --tag reproduction-01 --charts
```

For the prepared final matrix, use configs/bench/phase2.toml and a fresh campaign tag. A new run tests current software; it cannot reproduce historical scheduler state. Root operations create owned namespaces/veth/pins and namespace-local nftables state. Review privileges and kernel support before running; these commands were not invoked merely to regenerate this report.